

A Technical Design Document (TDD) or Solution Design Document (SDD) bridges the gap between business requirements and developer implementation. It acts as the blueprint for the system, ensuring all stakeholders align on *how* a problem will be solved technically before any code is written.

Here is an industry-standard template that covers all essential pointers required for a comprehensive solution design.

1. Document Control

Before diving into the technical details, track who is involved and how the document evolves.

- **Document History:** Version, Date, Author, and Description of Changes.
- **Approvers:** Names and roles of technical leads, architects, and product managers required to sign off.
- **Glossary:** Definitions of technical acronyms or business-specific jargon used in the document.

2. Executive Summary & Context

Set the stage for why this project exists and what it covers.

- **Problem Statement:** A brief summary of the business problem being solved.
- **Scope:** Clearly define what is **In-Scope** and (equally importantly) what is **Out-of-Scope** for this specific design phase.
- **Target Audience:** Who should read this document (e.g., frontend devs, backend devs, DevOps, QA).

3. High-Level Architecture

Provide a bird's-eye view of the system.

- **System Context:** How the new solution fits into the broader enterprise ecosystem.
- **Architecture Diagram:** A visual representation (e.g., C4 model, AWS architecture diagram) showing major components and their interactions.
- **Technology Stack:** The chosen languages, frameworks, databases, and cloud providers, along with brief justifications for these choices.

4. Component / Module Design

Break down the high-level architecture into actionable development blocks.

- **Microservices / Modules:** List each backend service, frontend application, or distinct module.
- **Responsibilities:** What each component does (e.g., "The User Service handles registration, login, and profile updates").
- **Component Interactions:** Sequence diagrams or flowcharts explaining how these components communicate to fulfill specific user journeys.

5. Data & Database Design

Explain how data will be stored, managed, and accessed.

- **Data Models / ER Diagram:** Entity-Relationship diagrams showing tables, collections, and relationships.
- **Schema Definitions:** Key database tables, primary/foreign keys, and indexes.
- **Data Flow:** How data enters the system, where it is processed, and where it rests.
- **Data Migration/Retention:** Plans for migrating legacy data or rules for archiving/deleting old data.

6. API & Integration Design

Detail the communication protocols.

- **Internal APIs:** REST, GraphQL, or gRPC endpoints between your own microservices. Include request/response payloads, headers, and error codes. (Often linked to a Swagger/OpenAPI spec).
- **External Integrations:** Details on connecting to third-party services (e.g., payment gateways, CRM systems), including webhook configurations and rate limit handling.
- **Event-Driven Communication:** If using message brokers (Kafka, RabbitMQ, SQS), list the topics, publishers, and subscribers.

7. Security & Compliance

Address how the system protects data and users.

- **Authentication & Authorization:** How users are verified (e.g., OAuth 2.0, JWT, SSO) and how their permissions are checked (Role-Based Access Control).
- **Data Security:** Encryption at rest (e.g., AES-256) and in transit (TLS 1.2+).
- **Compliance:** Adherence to regulatory standards like GDPR, HIPAA, or PCI-DSS (if applicable to the domain).
- **Secret Management:** How API keys and database credentials will be stored (e.g., AWS Secrets Manager, HashiCorp Vault).

8. Infrastructure & Deployment

Outline the physical or cloud topology required to run the code.

- **Cloud Resources:** Compute, storage, and networking resources needed (e.g., AWS EKS, S3, RDS).
- **Network Design:** VPCs, subnets, API Gateways, and load balancers.
- **CI/CD Pipeline:** The branching strategy and automated deployment process (e.g., GitHub Actions, Jenkins, Terraform for Infrastructure as Code).

9. Non-Functional Requirements (NFRs)

Define the system's operational standards.

- **Performance & Scalability:** Expected request load (RPS), latency targets, and auto-scaling triggers.
- **Availability & Reliability:** Uptime SLAs (e.g., 99.9%), failover strategies, and disaster recovery plans (Recovery Time Objective / Recovery Point Objective).

10. Observability (Logging, Monitoring, Alerting)

How the team will know if the system is healthy or broken.

- **Logging:** Structured logging format and aggregation tool (e.g., ELK stack, Datadog).
- **Metrics:** Key system metrics to track (CPU usage, DB connection pool, API latency).
- **Alerting:** Thresholds for triggering automated alerts to the on-call team via PagerDuty or Slack.

11. Assumptions, Dependencies & Risks

Document any external factors that could impact delivery.

- **Dependencies:** External teams, third-party APIs, or legacy systems that this solution relies on.
- **Assumptions:** Technical or business assumptions made during the design process (e.g., "Assuming peak traffic will not exceed 500 requests/second").
- **Risks & Mitigations:** Potential bottlenecks or failure points and how the team plans to address them.